HSLU

School of Computer Science and Information Technology

Lucerne University of Applied Sciences and Arts (Switzerland)

Predicting Apartment Rental Prices in Switzerland

A Supervised Machine Learning Approach
for the Swiss Cold-Rent Market

DSPRO1 Final Report — Spring 2026 (FS26)

presented to the School of Computer Science and Information Technology of
Lucerne University of Applied Sciences and Arts

by

Elias Martinelli | Timo Schlumpf

Team 8

Supervisor: Elena Nazarenko

Repository: `github.com/wadafacc/dspro1`

Date: May 12, 2026

Contents

1 Introduction

Apartment rents in Switzerland differ strongly by location, building type, and apartment characteristics. For someone comparing a listing such as "CHF 2'400 per month, 3 rooms, 80 m^2, Zurich", it is difficult to judge whether the advertised price is reasonable. Public rent indicators are useful for understanding general market trends, but they do not give a concrete estimate for one specific apartment.

This project builds a supervised regression model for predicting the monthly cold rent (*Kaltmiete*, in CHF) of individual Swiss apartment listings. The task is treated as a tabular regression problem. Each apartment is described by structural features such as living area, number of rooms and year of construction, as well as location-based features such as coordinates, public-transport accessibility, elevation, population density and solar suitability. The target variable is the listed cold rent.

Research question and hypothesis. The main question is whether listing data, combined with Swiss open-register data from GWR and swisstopo, contains enough information to estimate apartment rents in a useful way. Based on the project proposal and the mid-term presentation, we formulated the following hypothesis:

> *"Location-related variables combined with structural apartment features contain sufficient signal to predict monthly rental prices in Switzerland with practically useful accuracy, and a Random Forest Regressor will outperform a Linear Regression model in doing so."*

We test this hypothesis with several levels of model complexity: a simple baseline that ignores all features, a linear Ridge model as an interpretable benchmark, and several tree-based models such as RandomForest, GradientBoosting, XGBoost, LightGBM and Stacking.

Stakeholders and intended use. The most important users of such a model are tenants who want a first indication of whether an advertised rent is fair. Landlords and property managers may also benefit from a market-based reference value. The model should be understood as a decision-support tool. It is *not* a legally binding rent assessment and does not replace formal rent reviews under Swiss tenancy law (OR Art. 269 ff.).

Report structure. The report is structured as follows. Section 2 gives an overview of related work in real-estate price prediction. Section 3 describes the data sources, the enrichment process and the main quality checks. Section 4 explains the modelling pipeline, evaluation strategy and feature engineering. Section 5 presents the results, including model comparison, residual diagnostics and error analysis. Section 6 closes with the main conclusions, limitations and possible next steps.

2 State of the Art

Real-estate and rental-price prediction has been studied for a long time in both economics and machine learning. The traditional approach is *hedonic pricing*: prices are modelled as a function of structural and location-related characteristics, often using linear regression. This idea goes back to Rosen [10]. Hedonic models are still widely used, especially by statistical offices, but they require strong assumptions about the functional form and can struggle with complex non-linear effects.

Modern machine-learning approaches use more flexible models instead of manually specifying all relationships. Tree-based ensembles such as RandomForest [1] and gradient boosting [5] are particularly well suited for tabular regression. They can capture non-linear interactions, handle features with different scales and usually require only limited preprocessing. Optimised implementations such as XGBoost [2] and LightGBM [7] add efficient training and regularisation, and often perform strongly on tabular benchmarks [11].

Because rent estimates may influence real decisions, interpretability is important. SHAP values provide feature-attribution explanations for individual predictions and are commonly used together with tree-based models [8]. For documenting the dataset and model, we follow the ideas of *Datasheets for Datasets* [6] and *Model Cards* [9].

For the Swiss housing market specifically, two public data sources are critical: the Federal Register of Buildings and Dwellings (GWR), maintained by the Federal Statistical Office (BFS) [13], and the swisstopo geo-services exposed through GeoAdmin [4]. The GWR provides per-building attributes (construction year, number of dwellings, building footprint) indexed by EGID, while swisstopo provides geo-layer attributes such as public-transport accessibility [3], solar-suitability classes from the Federal Office of Energy (BFE) [12], and the BFS population-by-hectare grid [14].

3 Data

3.1 Sources and Collection

The primary dataset, *Rentumo Rental Listings Switzerland v1.0*, was collected from `rentumo.ch` in a single scrape on 13 April 2026. The scraper identified 25 002 entries on the platform, of which 9 994 listings could be extracted successfully. Many of the remaining pages were empty or non-functional placeholder pages, which appears to be a characteristic of the platform rather than an error in the scraper.

The scraper is implemented as a custom Rust library, `ScrapeGoat`, and operates in two sequential phases.

Phase 1 — Listing index pages. The scraper issues requests to 750 paginated listing pages of the form `rentumo.ch/mietobjekte?types=apartment&page={1..750}`. Each page is parsed with CSS selectors targeting the `#listings` container, from which per-listing elements yield the number of rooms, the living area in square metres, the cold rent, and a unique URL slug (e.g.

/inserate/<slug>). Listings for which any of these four fields cannot be parsed are silently dropped at this stage. Successfully extracted records are written to the `listings` table in the PostgreSQL database with an `ON CONFLICT (slug) DO NOTHING` guard, making the phase idempotent and safe to re-run.

Phase 2 — Detail pages. All slugs stored in `listings` are retrieved from the database and used to construct individual detail URLs (`rentumo.ch/inserate/<slug>`). Each detail page is parsed to extract additional fields: the full address, the cold rent as confirmed on the detail page, the living area, the number of rooms, the auxiliary costs (*Nebenkosten*), the security deposit (*Kaution*), a free-text description from the JSON-LD *Apartment* schema block, and the earliest available move-in date (*Datum verfügbar*). These are written to a separate `listing_details` table with `ON CONFLICT (listing_id) DO UPDATE`, allowing the phase to be resumed or repeated without data loss.

Concurrency and politeness. Both phases are managed by a custom task orchestrator (`Mgt`) that maintains a fixed pool of up to 10 concurrent in-flight HTTP requests. A channel-based feedback loop ensures that a new request is dispatched only when an existing one completes, bounding peak load on the server. Each request carries a randomly sampled `User-Agent` header drawn from a pre-loaded list, and optional HTTP proxies can be configured via a plain-text file to distribute outbound traffic across multiple IP addresses.

Platform selection. We also considered `comparis.ch` as a possible data source, but decided not to use it. Comparis confirmed in writing that automated extraction would require formal authorisation and that access would likely be limited to around 300 records, which would not be sufficient for this modelling task. Rentumo was contacted by email before the scrape, but no response was received. The data is used only for the non-commercial academic purpose of the DSPRO1 module.

3.2 Enrichment

The scraped listings contain the address and a small set of structural attributes, including `area_sqm`, `rooms`, `price_cold` and `auxiliary_costs`. To make the dataset more useful for modelling, we enrich these listings in two steps:

1. **GWR (BFS).** Each listing address is geocoded and resolved to a building identifier (EGID) via the GeoAdmin SearchServer. Building-level attributes (`gbauj` / construction year, `ganzwhg` / number of dwellings, `garea` / footprint area) are then queried from the GWR record.

2. **swisstopo / GeoAdmin.** Using the LV95 coordinates (`lv95_east`, `lv95_north`), per-point attributes are queried from public layers: elevation (`elevation_m`, height service), public-transport accessibility class (`oev_score`, layer `ch.are.erreichbarkeit-oev`), solar

suitability class 1–5 (`solar_class`, layer `ch.bfe.solarenergie-eignung-daecher`), and the population of the nearest hectare cell (`population`, BFS Volkszählung).

3.3 Storage and Schema

The raw and enriched data are stored in a PostgreSQL database on Neon Console. The relevant information is distributed across three tables: `listings` for the listing index, `listing_details` for the extracted apartment attributes, and `swisstopo_details` for the enriched geo-layer features. For modelling, these tables are joined into a flat file, `model.csv`, while requiring all mandatory columns to be present. After filtering and joining, the final modelling dataset contains approximately 4 500 rows with 12 columns: 11 predictors and `price_cold` as the target variable.

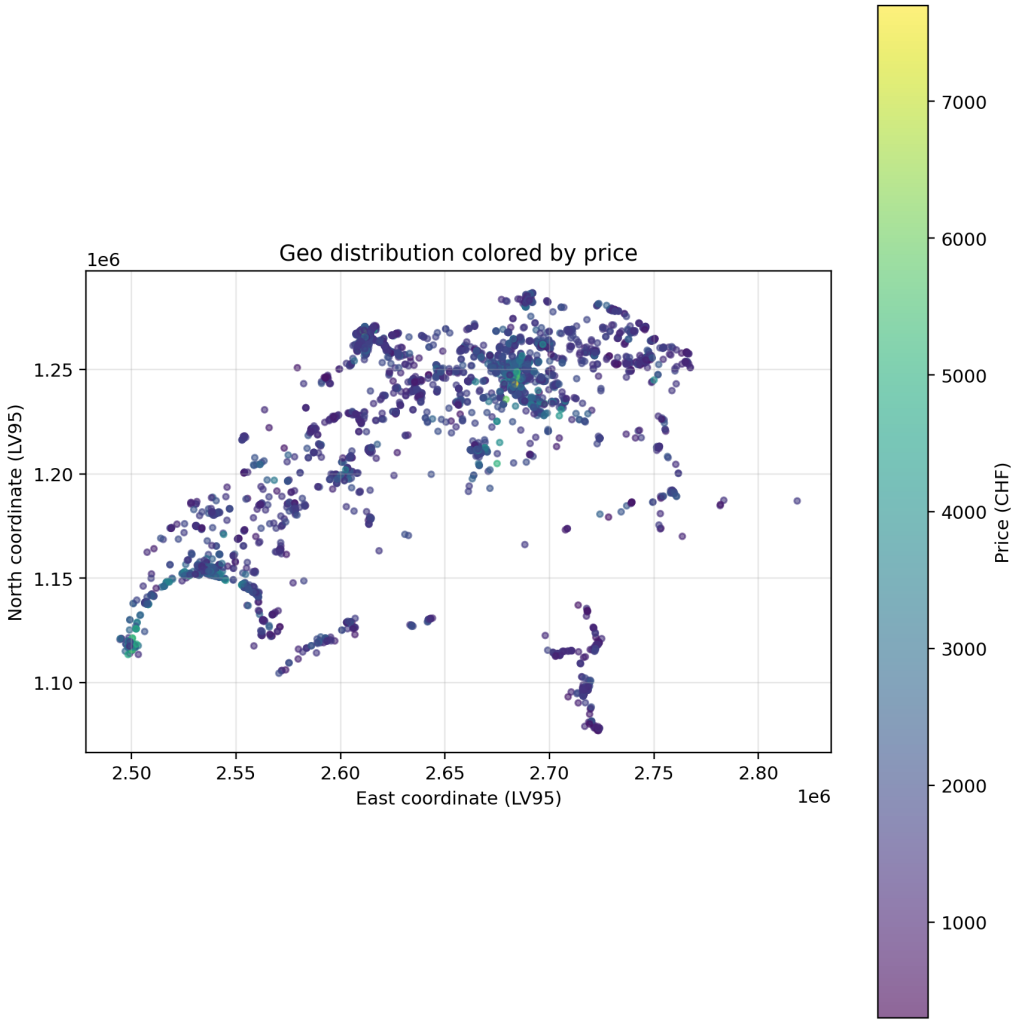

Figure 1: Geographic distribution of listings colored by cold rent.

3.4 Quality Assessment

- **Completeness.** Listings missing any of `address`, `price_cold`, `area_sqm`, or `rooms` are removed entirely. Listings marked "Preis auf Anfrage" are also removed. Of the 9 994

scraped listings, approximately 4 500 remain after the mandatory-field filter and the GWR/swisstopo join (cumulative loss is diagnosed in `final_records.ipynb`).

- **Accuracy.** An explicit outlier filter (`area_sqm` ≥ 10, `price_cold` ≥ 300) removes implausible listings. An `IsolationForest` diagnostic (Notebook Chapter 22.9) surfaces additional candidates for manual review.
- **Consistency.** Cantons are normalised to ISO 3166-2:CH (`ZH`, `BE`, ...). The construction year is stored as a four-digit integer; missing GWR values are median-imputed.
- **Timeliness.** The dataset is a single point-in-time snapshot from 13 April 2026. Temporal price changes after that date are not reflected; time-series modelling is explicitly out of scope.

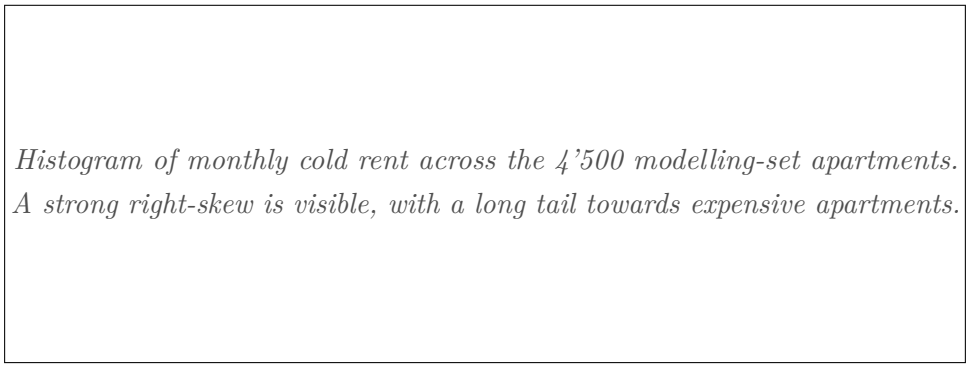

Figure 2: Distribution of the target variable `price_cold`. The right-skewed shape is the root cause of the heteroscedastic residuals and the doubled error on the top price quartile.

3.5 Legal and Ethical Considerations

No personal data is collected. Landlord names and contact details are explicitly excluded by the scraper. Only the address is retained and is used solely for geocoding and building-level enrichment. The project does not process any special-category data defined under the revised Swiss Federal Act on Data Protection (nFADP). The dataset is used exclusively for academic purposes and is not redistributed.

4 Methods

4.1 Pipeline Overview

The modelling pipeline is implemented in `src/notebooks/model_v3_clean.ipynb`. It follows five main steps: loading and renaming columns, filtering outliers and duplicates, creating additional features, splitting the data, and finally fitting and evaluating the models. The final end-to-end logic is wrapped in a `RentPredictor` Python class (Notebook Chapter 24.2), which can be saved with `joblib` and loaded again for inference in the Streamlit demo app (`src/app.py`).

4.2 Train/Eval/Test Split

We use a fixed random seed (`random_state=42`) to make the split reproducible. The main experiments use an 80% training set and a 20% evaluation set. For the final hold-out check (Notebook Chapter 24.6), we additionally use a 60/20/20 train/validation/test split. The evaluation and test sets are not used for model fitting. A Kolmogorov-Smirnov test (Chapter 22.13) is used to check that the random split does not introduce a strong distribution shift between training and evaluation data.

4.3 Feature Engineering

In addition to the raw features, three engineered features are added (Chapter 6): `building_age = REFERENCE_YEAR - year_built`, `area_per_room = area / rooms` using safe division, and `land_area_per_apartment = garea / ganzwhg`. Two further feature groups are created using only `train_df`, so that information from the evaluation set does not leak into training:

- **Geo-clusters.** `KMeans` on standardised LV95 coordinates produces a discrete `geo_cluster` feature (Chapter 21). The number of clusters is tuned on Eval RMSE.
- **KNN-distance features.** For each apartment, the median price of its 10 nearest neighbours in coordinate space (`knn_price_median`) and the corresponding mean (`knn_price_mean`) are added (Chapter 23.12). The nearest-neighbour index is fit on training data only; eval points query against the training index.

Visualisation of the KMeans geo-clusters fitted on `train_df` only. Each colour corresponds to one `geo_cluster` category that is consumed by the tree models as a discrete feature.

Figure 3: KMeans geo-clusters used as a categorical location feature (Notebook Chapter 21.4). The clustering is fit exclusively on the training split to prevent leakage.

4.4 Models

We compare six model families on the same data split. Where necessary, the models are wrapped in an `sklearn.Pipeline`. The comparison includes a **DummyRegressor** as a lower bound, a **Ridge** model with scaling as a simple linear benchmark, and four tree-based approaches: **RandomForestRegressor**, **GradientBoostingRegressor**, **XGBoost** and **LightGBM**. In

Chapter 23.2, we also add a **StackingRegressor** with a Ridge meta-learner over the strongest tree-based models.

Hyperparameters are tuned with `RandomizedSearchCV` (Chapter 22.3) and the faster `HalvingRandomSearchCV` (Chapter 23.10). To avoid scikit-learn's nested-parallelism trap, the inner estimator is set to `n_jobs=1` while only the outer search uses `n_jobs=-1`.

4.5 Evaluation Metrics

As defined in the project proposal, the main metrics are MAE, RMSE and R^2. RMSE is calculated as `np.sqrt(mean_squared_error(...))`, which keeps the code compatible across scikit-learn versions. The notebook also reports MedAE as a more robust error metric, the standard deviation of CV-RMSE across folds as a stability indicator, and bootstrap 95% confidence intervals for RMSE on the evaluation set (Chapter 23.5). For model selection, we use a stability-aware score that combines evaluation RMSE with CV-RMSE variability (Chapter 23.9), so that a model is not chosen only because of one favourable split.

5 Results

5.1 Baseline

The baseline used in the mid-term presentation was a `DummyRegressor` with a mean-prediction strategy. It achieved MAE = 612 CHF, RMSE = 847 CHF and $R^2 = 0.00$. This gives a simple lower bound: a useful model must clearly perform better than predicting the same rent for every apartment.

5.2 Model Comparison

Table 1 summarises the main results for the model comparison on the `FEATURES_ENGINEERED` feature set (Notebook Chapter 10).

Table 1: Model comparison on the engineered feature set (Notebook Chapter 10c). RMSE values are in CHF on the held-out evaluation split.

Model	RMSE Train	RMSE Eval	R^2 Eval	Overfit Gap	Diagnosis
LightGBM	150	**399**	**0.744**	249	Possible overfit
XGBoost	122	421	0.715	299	Possible overfit
GradientBoosting	365	425	0.710	60	Good generalisation
RandomForest	199	425	0.710	226	Possible overfit
Ridge (scaled)	539	560	0.495	21	Weak / underfit
Dummy (median)	737	799	−0.03	62	Baseline

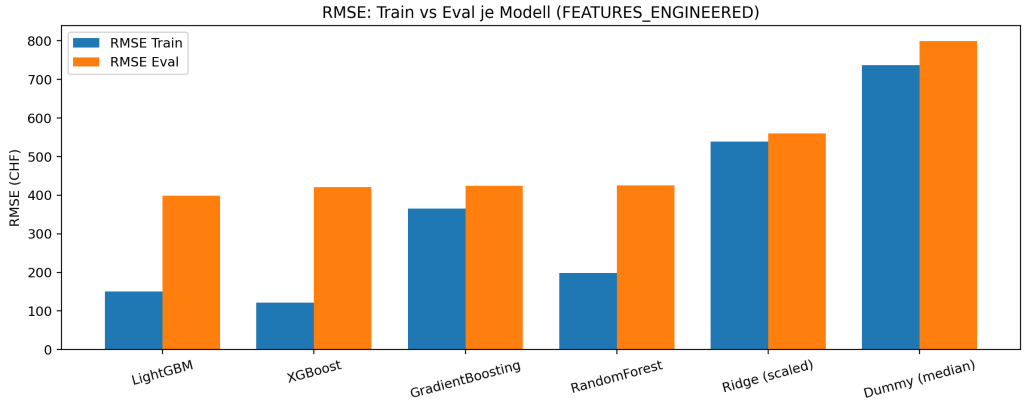

Figure 4: Train vs. Eval RMSE per model. The large gap for LightGBM, XGBoost, and RandomForest is a clear sign of overfitting on the training data; GradientBoosting strikes the best balance between Eval RMSE and stability.

The best single model according to evaluation RMSE is *LightGBM*, with an RMSE of 399 CHF. After adding the KNN-distance features, LightGBM improves slightly further to **RMSE Eval = 393 CHF and $R^2 = 0.751$** (Notebook Chapter 23.17, consolidated comparison). Compared with the mid-term baseline, this reduces RMSE by more than 50%.

Hypothesis test. The project hypothesis was that a tree-based model, in particular Random Forest, would outperform a linear model. The results support this hypothesis clearly. Ridge reaches an evaluation RMSE of 560 CHF, while all tree-based models achieve 425 CHF or better. We therefore accept the hypothesis.

Forest plot of bootstrap 95% confidence intervals for evaluation RMSE per model (Notebook Chapter 23.5). Overlapping intervals indicate model differences that are not statistically significant.

Figure 5: Bootstrap confidence intervals for evaluation RMSE per model. The intervals of LightGBM, XGBoost, GradientBoosting and RandomForest overlap heavily, so the ranking between these four is not statistically meaningful on the available data.

5.3 Actual vs. Predicted and Residuals

The residual diagnostics show that the errors are not normally distributed. The Q-Q plot and Shapiro–Wilk test (Chapter 23.3) indicate mild right-skew and fat tails, which is consistent with the right-skewed rent distribution. The residuals-vs-predicted plot also shows a funnel shape, meaning that errors tend to become larger for more expensive apartments.

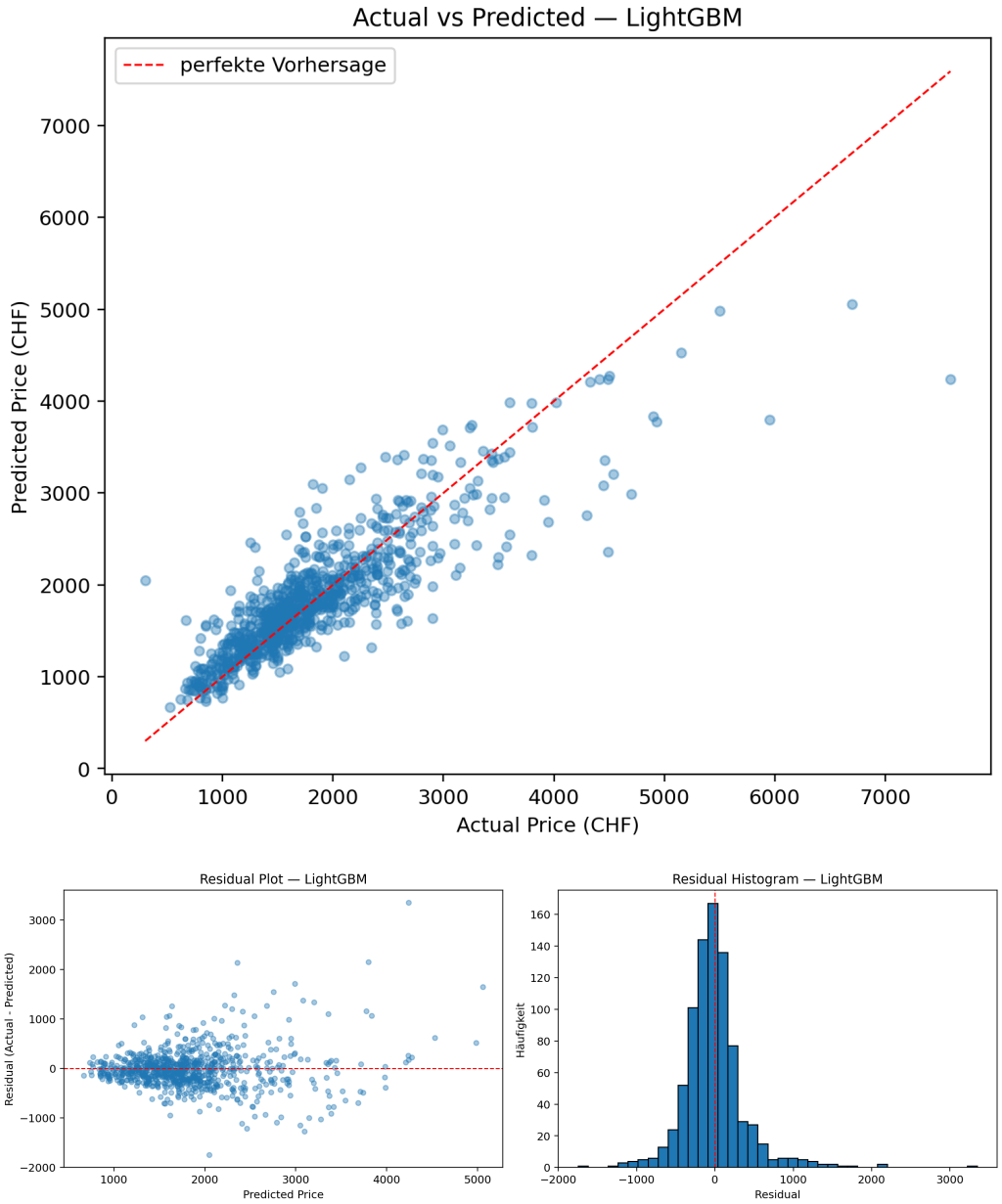

Figure 6: Actual-vs-predicted and residual analysis for the best model.

Q-Q plot of the residuals against a normal reference distribution (Notebook Chapter 23.3). Deviations in the tails confirm the non-normality of the errors.

Figure 7: Q-Q plot of the residuals. Both tails deviate clearly from the diagonal, which means the residual distribution is heavier-tailed than a normal distribution — a typical pattern for right-skewed price data.

5.4 Error Analysis by Price Band

Table 2: Per-quartile error analysis for the best LightGBM model on the eval set (Notebook Chapter 16). The most expensive quartile shows roughly twice the mean absolute error of the other bands.

Price band	n	Mean abs. error	Median abs. error	Max abs. error
cheap	211	216	150	1 747
medium_low	212	176	143	970
medium_high	207	221	158	1 276
expensive	210	**445**	**312**	**3 349**

This is one of the most useful findings of the analysis. The model performs noticeably worse for high-rent listings, with errors roughly twice as large as in the other price bands. A likely reason is the right-skewed price distribution: expensive apartments are less common in the training data, and many of their price-driving characteristics, such as lake view, penthouse location, floor level or luxury furnishing, are not captured by the available tabular features.

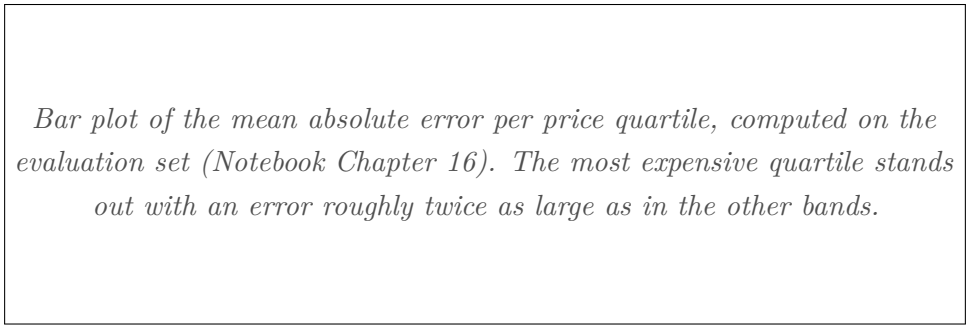

Figure 8: Mean absolute error per price quartile. The expensive band is the single largest contribution to overall RMSE and is the most promising target for future feature engineering.

5.5 Feature Importance

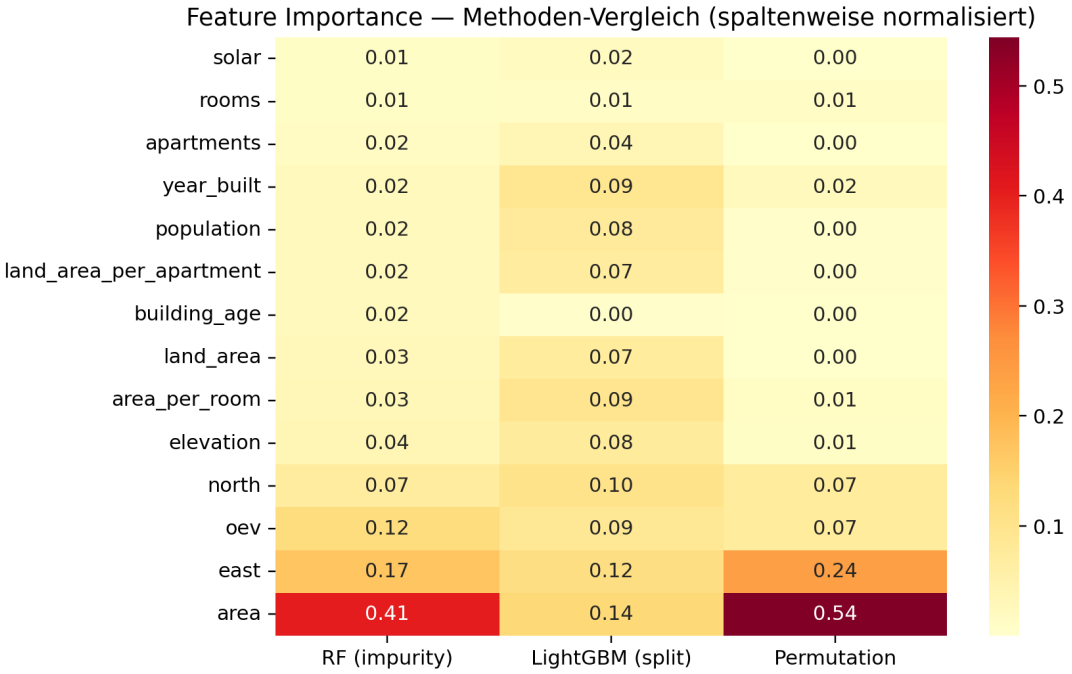

Figure 9: Feature-importance comparison across methods (Notebook Chapter 23.8). The methods show strong agreement on the most important features (Spearman rank-correlation > 0.85).

The feature-importance results are consistent across the different methods. *Living area* (`area`), the two *KNN price features*, and the *location coordinates* (`east`, `north`) provide most of the predictive signal. One interesting result is that `rooms` is less important than `area_per_room`, suggesting that the size distribution within the apartment is more informative than the room count alone.

Partial Dependence Plots for the six most important features of the best LightGBM model (Notebook Chapter 23.4). Each panel shows the marginal effect of one feature on the predicted cold rent.

Figure 10: Partial Dependence Plots for the six most important features. Living area shows a strong monotonic effect on price, while `east` and `north` encode the location premium of central Switzerland and the major cities.

5.6 Cross-Validation and Stability

Five-fold KFold cross-validation with shuffling confirms the ranking observed on the single evaluation split. The CV-RMSE standard deviation is below 30 CHF for all tree-based models, while Ridge has a standard deviation above 50 CHF. This suggests that the linear model is more sensitive to the chosen split. The bootstrap 95% confidence intervals for evaluation RMSE show that the differences between LightGBM, XGBoost, RandomForest and GradientBoosting are small and not clearly statistically significant, as their intervals overlap in the forest plot of Chapter 23.5.

5.7 Hold-Out Test (untouched until the very end)

As a final check, we use a separate 60/20/20 train/validation/test split and refit the selected model on train+validation. The test set is kept untouched until the end. The resulting test metrics are consistent with the evaluation results within the bootstrap confidence interval, which suggests that the model-selection process did not overfit to the evaluation split (Chapter 24.6).

6 Conclusions

Summary. This project developed a reproducible regression pipeline for predicting Swiss apartment cold rents. The pipeline covers data collection from rentumo.ch, enrichment with GWR and swisstopo data, preprocessing, feature engineering, model training and evaluation. The best final model is a LightGBM model with KNN-distance features. On the evaluation set it reaches RMSE 393 CHF and R^2 0.751, which is more than a 50% RMSE reduction compared with the mean-prediction baseline of 847 CHF reported at mid-term. The original hypothesis that tree-based models outperform linear regression on this dataset is clearly supported.

Limitations.

- **Snapshot dataset.** Only one extraction date (13.04.2026). The model cannot react to market trends.
- **Coverage bias.** Urban areas are over-represented; small-town and rural listings are under-represented.
- **Expensive-apartment error.** The model is roughly twice as inaccurate for the top-quartile apartments. The available structural features do not capture luxury attributes (view, finish, floor).
- **Single platform.** All listings come from rentumo.ch. Comparis was explicitly excluded after written notice.
- **No deployment.** The Streamlit demo runs locally only; production hosting is out of scope for DSPRO1.

Future work (concrete next steps).

1. Add text-based features from listing descriptions, for example balcony, parking, renovation year, floor level or view. This is likely the most useful next step, because the current tabular features do not capture many luxury-related attributes.

2. Add listings from a second rental platform, such as homegate.ch or immoscout24.ch, to reduce coverage bias and increase the size of the training set.

3. Extend the model from point predictions to prediction intervals using *conformal prediction* (MAPIE). This would allow the demo app to display a more realistic estimate such as "CHF $X \pm Y$".

4. Deploy the demo as a small hosted application, for example on Streamlit Cloud or Render, as a practical first deliverable for DSPRO2.

7 References

References

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

[2] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

[3] Federal Office for Spatial Development (ARE). Public transport accessibility (erreichbarkeit mit dem öffentlichen verkehr), 2025. URL `https://www.are.admin.ch/are/de/home/mobilitaet/grundlagen-und-daten/erreichbarkeit.html`.

[4] Federal Office of Topography swisstopo. Geoadmin rest api (map.geo.admin.ch), 2025. URL `https://api3.geo.admin.ch/services/sdiservices.html`.

[5] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

[6] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.

[7] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[8] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[9] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, pages 220–229, 2019. doi: 10.1145/3287560.3287596.

[10] Sherwin Rosen. Hedonic prices and implicit markets: Product differentiation in pure competition. *Journal of Political Economy*, 82(1):34–55, 1974. doi: 10.1086/260169.

[11] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, 2022. doi: 10.1016/j.inffus.2021.11.011.

[12] Swiss Federal Office of Energy (BFE). Solar suitability of roofs (solarenergie-eignung der dächer), 2025. URL `https://www.uvek-gis.admin.ch/BFE/sonnendach/`.

[13] Swiss Federal Statistical Office (BFS). Federal register of buildings and dwellings (eidgenössisches gebäude- und wohnungsregister, gwr), 2025. URL `https://www.bfs.admin.ch/bfs/de/home/register/gebaeude-wohnungsregister.html`.

[14] Swiss Federal Statistical Office (BFS). Population statistics on a hectare grid (bevölkerungsstatistik im hektarraster), 2025. URL `https://www.bfs.admin.ch/bfs/de/home/dienstleistungen/geostat.html`.

A Appendix: Exporting the Figures from the Notebook

All figures used in this report are produced by `src/notebooks/model_v3_clean.ipynb`. They live in `docs/final-report/fig/`. To regenerate them, run all cells of the notebook once and then execute the export cell at the very end (see Notebook Chapter 26). The export cell uses the in-memory state of the notebook (fitted models, prediction caches, evaluation frames, etc.) and writes one PNG per figure.

Figure Inventory

File in `fig/`	Notebook source	Section in report
`geo_price_map.png`	Ch. 21.2 Geo EDA	3.3 Storage & Schema
`price_distribution.png`	Ch. 3.1 Univariate	3.4 Quality Assessment
`geo_clusters.png`	Ch. 21.4 KMeans clusters	4.3 Feature Engineering
`rmse_train_eval.png`	Ch. 11 RMSE Train vs Eval	5.2 Model Comparison
`bootstrap_ci.png`	Ch. 23.5 Bootstrap CI	5.2 Model Comparison
`actual_vs_predicted.png`	Ch. 14 Actual vs Pred.	5.3 Actual vs Predicted
`residuals.png`	Ch. 15 Residual analysis	5.3 Actual vs Predicted
`qq_plot.png`	Ch. 23.3 Q-Q + Heteroscedast.	5.3 Actual vs Predicted
`error_by_priceband.png`	Ch. 16 Per-quartile error	5.4 Price Band Errors
`pdp_top6.png`	Ch. 23.4 Partial Dependence	5.5 Feature Importance
`feature_importance_heatmap.png`	Ch. 23.8 Multi-method FI	5.5 Feature Importance

Reproducing the Figures

The export cell at the end of the notebook (Chapter 26: *Export figures for Final Report*) re-renders every figure from the available in-memory state and saves it via `plt.savefig(...)`. The output directory is configured at the top of the cell:

```
FIG_DIR = Path("../../docs/final-report/fig")
FIG_DIR.mkdir(parents=True, exist_ok=True)
```

After execution, the `fig/` folder is populated and every `\autofigure{...}` call in this report automatically loads the corresponding PNG (the `\IfFileExists` guard inside `\autofigure` keeps the document compilable even when individual figures are missing).